

UNIVERSIDAD DE CUENCA

Facultad de Ingeniería

Computación

Realizado por:

Joseline Banegas

Maria Lojano

Adrian Muñoz

Documento de Especificación y Diseño: Implementación del Estándar de Firma Digital de Curva Elíptica (ECDSA) en C para CryptoLib

---

1. Introducción

El **Algoritmo de Firma Digital de Curva Elíptica (ECDSA, *Elliptic Curve Digital Signature Algorithm*)** es la variante sobre curvas elípticas del algoritmo clásico de firma digital (DSA). Estandarizado por el Instituto Nacional de Estándares y Tecnología de EE. UU. (**NIST**) en el documento **FIPS PUB 186-5**, ECDSA proporciona los mismos servicios de seguridad criptográfica que RSA o DSA —**autenticidad, integridad de datos y no repudio**—, pero con una eficiencia computacional y de almacenamiento significativamente superior.

Mientras que esquemas tradicionales como RSA o DSA sobre campos finitos requieren claves de 2048 o 3072 bits para ofrecer un nivel de seguridad de 128 bits, ECDSA alcanza este mismo nivel de resistencia criptográfica utilizando claves de tan solo **256 bits** (por ejemplo, sobre la curva estándar P-256 o secp256k1). Esto se fundamenta en la dificultad computacional del **Problema del Logaritmo Discreto en Curvas Elípticas (ECDLP)**, para el cual no se conocen algoritmos de tiempo subexponencial.

El presente proyecto aborda el análisis matemático, diseño modular e implementación en lenguaje **C** del estándar **ECDSA**, integrándolo dentro de la librería criptográfica **CryptoLib** mediante el aprovechamiento de su infraestructura de aritmética modular de precisión arbitraria (BigInt) y funciones resumen (*Hash*).

2. Objetivos

2.1. Objetivo General

Diseñar, implementar y validar en lenguaje C el algoritmo de firma digital **ECDSA** siguiendo estrictamente los lineamientos normativos del estándar **NIST FIPS PUB 186-5**, integrándolo como un

nuevo módulo funcional dentro de la librería **CryptoLib**.

2.2. Objetivos Específicos

1. **Analizar la documentación normativa del NIST** (FIPS 186-5, SP 800-186 y FIPS 180-4 / FIPS 202) para extraer las especificaciones matemáticas de los parámetros de dominio, aritmética de curva y algoritmos de firma/verificación.
2. **Implementar la capa de aritmética geométrica sobre curvas elípticas en F_p** en lenguaje C (adición de puntos ec_point_add, duplicación de puntos ec_point_double y multiplicación escalar ec_scalar_mul) utilizando las primitivas BigInt de CryptoLib.
3. **Desarrollar las funciones criptográficas principales de ECDSA**: generación de pares de claves (ecdsa_keygen), generación de firma digital (ecdsa_sign) y verificación de firma (ecdsa_verify).
4. **Validar la implementación en C** mediante vectores de prueba oficiales del NIST (*Cryptographic Algorithm Validation Program - CAVP*) para garantizar la interoperabilidad y exactitud matemática del módulo.

3. Documentación Normativa (Estándares del NIST y Referencias Técnicas)

Para garantizar que el código en C cumpla con los estándares internacionales, el desarrollo se rige por los siguientes documentos oficiales:

Documento / Estándar	Título Oficial	Secciones Clave para la Implementación en Código
1. NIST FIPS PUB 186-5 *(Febrero 2023)*	*Digital Signature Standard (DSS)*	• **Sección 6.2.1 (Pág. 21):** Generación del par de claves (d, Q). • **Sección 6.4.1 (Pág. 25):** Algoritmo de generación de firma (r, s). • **Sección 6.4.2 (Pág. 27):** Algoritmo de verificación de firma. • **Apéndice B.2 (Pág. 49):** Cálculo del inverso multiplicativo modular (k^{-1} mod n). Enlace: Descargar PDF Oficial (NIST FIPS 186-5)

Documento / Estándar	Título Oficial	Secciones Clave para la Implementación en Código
2. NIST SP 800-186 *(Febrero 2023)*	*Recommendations for Discrete Logarithm-based Cryptography: Elliptic Curve Domain Parameters*	• **Sección 3.2.1.3 (Pág. 16):** Parámetros hexadecimales oficiales de la curva **Curve P-256** (secp256r1): primo p, coeficientes a, b, punto base G = (Gx, Gy) y orden primo n. • **Apéndice B:** Fórmulas de ley de grupo para curvas de Weierstrass cortas. Enlace: Descargar PDF Oficial (NIST SP 800-186)
3. NIST FIPS PUB 180-4 / FIPS 202	*Secure Hash Standard (SHS) / SHA-3 Standard*	• Especificación de la función resumen Hash(M) encargada de digerir el mensaje de entrada antes de firmar y verificar. Enlace: Descargar PDF Oficial (NIST FIPS 180-4)
4. Bibliografía de Referencia para C	*Guide to Elliptic Curve Cryptography* (Hankerson, Menezes, Vanstone)	• **Capítulo 3 (Algoritmos 3.1 - 3.30):** Pseudocódigo de adición afín y multiplicación escalar binaria (*Double-and-Add*). • **Capítulo 4 (Algoritmos 4.29 - 4.30):** Pseudocódigo estructurado de ECDSA.

4. Marco Teórico y Desarrollo Matemático

4.1. Parámetros de Dominio de la Curva Elíptica

Una curva elíptica en forma corta de Weierstrass definida sobre un campo finito primo F_p (donde $p > 3$ es primo) está dada por el conjunto de puntos $(x, y) \in F_p \times F_p$ que satisfacen la ecuación:

$$E(F_p) : y^2 \equiv x^3 + a \cdot x + b \pmod{p}$$

junto con un punto especial llamado **punto en el infinito (O)**, que actúa como el elemento neutro de la suma. Los parámetros de dominio están formados por la tupla (p, a, b, G, n, h):

- **p:** Número primo que define el tamaño del campo finito F_p.
- **a, b $\in F_p$:** Coeficientes de la ecuación de la curva tales que $4a^3 + 27b^2 \neq 0 \pmod{p}$.
- **G = (x_G, y_G):** Punto generador base de la curva de orden primo n.
- **n:** Orden del punto G (el menor entero positivo tal que $[n]G = O$).
- **h:** Cofactor de la curva (en curvas estándar como P-256 o secp256k1, h = 1).

4.2. Ley de Grupo: Suma y Duplicación de Puntos en Coordenadas Afines

Sean $P = (x_1, y_1)$ y $Q = (x_2, y_2)$ dos puntos en la curva $E(F_p)$. Para calcular $R = P + Q = (x_3, y_3)$:

1. **Casos con el punto en el infinito (O):**
   - $O + P = P$ y $P + O = P$.
   - Si $x_1 = x_2$ y $y_1 \equiv -y_2 \pmod p$, entonces $P + Q = O$.
2. **Adición de dos puntos distintos ($P \neq Q$, es decir, $x_1 \neq x_2$):**

   La pendiente λ de la recta secante que pasa por P y Q se calcula como:

   $\lambda = (y_2 - y_1) \cdot (x_2 - x_1)^{-1} \bmod p$
3. **Duplicación de un mismo punto ($P = Q$, con $y_1 \neq 0$):**

   La pendiente λ de la recta tangente en P se calcula como:

   $\lambda = (3x_1^2 + a) \cdot (2y_1)^{-1} \bmod p$
4. **Cálculo de las coordenadas resultantes (x_3, y_3) (para ambos casos):**

   $x_3 = (\lambda^2 - x_1 - x_2) \bmod p$

   $y_3 = (\lambda(x_1 - x_3) - y_1) \bmod p$

4.3. Multiplicación Escalar (Q = [k]P)

Dado un entero k de 256 bits y un punto P, la operación $[k]P = P + P + ... + P$ (k veces) se calcula en tiempo polinómico $O(\log_2 k)$ mediante el método binario **Double-and-Add** (recorriendo los bits de k desde el más significativo al menos significativo):

- Se inicializa el acumulador $R = O$.
- Por cada bit k_i de k:
  1. $R \leftarrow 2R$ (ec_point_double)
  2. Si el bit $k_i == 1$, entonces $R \leftarrow R + P$ (ec_point_add).

4.4. Los 3 Algoritmos del Estándar ECDSA (NIST FIPS 186-5, Sección 6)

A. Generación de Par de Claves (Sección 6.2.1)

1. Seleccionar un número entero pseudoaleatorio d en el intervalo [1, n - 1].
2. Calcular el punto de clave pública $Q = (x_Q, y_Q) = [d]G$ mediante multiplicación escalar.
3. **Salida:** Clave privada d, Clave pública Q.

B. Generación de Firma Digital (Sección 6.4.1)

Para firmar un mensaje M con la clave privada d:

1. Calcular el resumen criptográfico: H = Hash(M), y convertirlo a un entero e (tomando los len(n) bits más significativos si el hash es más largo que n).
2. Generar un entero efímero aleatorio (o determinista según RFC 6979) $k \in [1, n - 1]$.
3. Calcular el punto de curva $(x_1, y_1) = [k]G$.
4. Calcular $r = x_1 \bmod n$. Si $r = 0$, volver al paso 2.
5. Calcular el inverso modular $k^{-1} \bmod n$ y evaluar:
   $s = k^{-1} \cdot (e + d \cdot r) \bmod n$Si $s = 0$, volver al paso 2.
6. **Salida:** La firma digital es el par de enteros (r, s).

C. Verificación de Firma Digital (Sección 6.4.2)

Para verificar la firma (r, s) sobre un mensaje M utilizando la clave pública Q:

1. Comprobar que $r, s \in [1, n - 1]$. Si no se cumple, **rechazar la firma**.
2. Calcular H = Hash(M) y convertirlo al entero e.
3. Calcular el inverso modular de s: $w = s^{-1} \bmod n$.
4. Calcular los escalares $u_1 = (e \cdot w) \bmod n$ y $u_2 = (r \cdot w) \bmod n$.
5. Calcular el punto de curva $X = (x_1, y_1) = [u_1]G + [u_2]Q$.
6. Si X = O (punto en el infinito), **rechazar la firma**.
7. Calcular $v = x_1 \bmod n$.
8. **Validación final:** La firma es **VÁLIDA** si y solo si v == r.

5. Diseño de Arquitectura e Integración en CryptoLib (Lenguaje C)

Para integrar ECDSA limpiamente dentro de CryptoLib, el módulo se divide en dos archivos (ecdsa.h y ecdsa.c) que consumen la aritmética BigInt existente en la librería.

5.1. Estructuras de Datos Propuestas (ecdsa.h)

```
#ifndef CRYPTOLIB_ECDSA_H
```

```
#define CRYPTOLIB_ECDSA_H

#include "libcrypt.h" /* Cabecera principal de CryptoLib (BigInt, Hash, etc.) */

/* 1. Representacion de un punto (x, y) en coordenadas afines sobre F_p */
typedef struct {
    BigInt x;
    BigInt y;
    int is_infinity; /* 1 si representa el punto en el infinito (O), 0 en caso contrario */
} ECPoint;

/* 2. Parametros de dominio de la curva eliptica (ej. NIST P-256) */
typedef struct {
    BigInt p;       /* Primo del campo finito F_p */
    BigInt a;       /* Coeficiente 'a' de la curva */
    BigInt b;       /* Coeficiente 'b' de la curva */
    ECPoint G;      /* Punto generador base G = (Gx, Gy) */
    BigInt n;       /* Orden primo del punto generador G */
} ECCurve;

/* 3. Estructura de la Firma Digital ECDSA */
typedef struct {
    BigInt r;
    BigInt s;
} ECDSASignature;

/* Prototipos de funciones del modulo */
void ec_init_p256(ECCurve *curve);
void ec_point_add(const ECCurve *curve, const ECPoint *P, const ECPoint *Q, ECPoint *R);
void ec_point_double(const ECCurve *curve, const ECPoint *P, ECPoint *R);
void ec_scalar_mul(const ECCurve *curve, const BigInt k, const ECPoint *P, ECPoint *R);

int ecdsa_keygen(const ECCurve *curve, BigInt priv_d, ECPoint *pub_Q);
int ecdsa_sign(const ECCurve *curve, const BigInt priv_d, const BigInt hash_e, ECDSASignature *sig);
int ecdsa_verify(const ECCurve *curve, const ECPoint *pub_Q, const BigInt hash_e, const ECDSASignature
*sig);
```

```
#endif /* CRYPTOLIB_ECDSA_H */
```

6. Plan de Implementación Paso a Paso (Hoja de Ruta hacia el Código)

1. **Fase 1: Inicialización de Constantes (ec_init_p256)**
    - Cargar en memoria BigInt los valores hexadecimales de la curva estándar **NIST P-256** (extraídos de *NIST SP 800-186*, Sección 3.2.1.3).
2. **Fase 2: Aritmética de Puntos (ec_point_double, ec_point_add, ec_scalar_mul)**
    - Programar las ecuaciones de la pendiente λ usando las funciones modulares de CryptoLib.
    - **Prueba de control 1:** Verificar que [1]G = G, que [2]G calculado con point_double coincida con el vector oficial de 2G, y que [n]G = O (punto en el infinito).
3. **Fase 3: Primitivas de Firma y Verificación (ecdsa_keygen, ecdsa_sign, ecdsa_verify)**
    - Codificar la generación de (r, s) y la comprobación v == r.
    - **Prueba de control 2:** Firmar un hash de prueba con un k fijo conocido del NIST CAVP y comprobar que r y s coinciden bit a bit con el PDF del estándar.
4. **Fase 4: Integración en el Makefile de CryptoLib y CLI de Pruebas**
    - Compilar ecdsa.o dentro de la librería y crear un ejecutable test_ecdsa.c que demuestre generación de clave, firma de un mensaje real y detección de alteraciones si el mensaje cambia.